CHS — COMPLETE HOUSING SOLUTIONS

SYSTEM DOCUMENTATION

Real estate super-app — Nigeria

Document prepared: September 2026 — this edition
Compiled from the project's live handover records, source code, and database migration history

October 2026 Update — read this first

This update is inserted at the front of the September 2026 System Documentation. Where it disagrees with the pages that follow, THIS UPDATE IS CORRECT. Figures were read from the live database and the repository on 9 October 2026.

Current figures

Item	September edition	Now (9 October 2026)
Migrations recorded live	290	363 (latest 20261008161505)
SQL records in backend-v2	—	395 files, numbered up to 475 (with notes 475c–475j)
Public database functions	443	602
Public tables	—	115
Terms & Conditions version	3	6 (app and database)
Feature catalog	230 entries, 16 sections	247 entries, 17 sections

What changed since the September edition

- **Hotel, lodge, shortlet and event-centre booking** rebuilt as request-first: room units with a database-level double-booking guard, host calendar console, everything between guest and host relayed by CHS with message review, Instant Confirm and Express lanes, peak pricing, host cancellation strikes (warning, 10% penalty, suspension within 90 days) and digital check-in (arrival pass, registration card, lockbox code, front desk, no-show handling).
- **Wallet security**: every payment on the platform is made through the CHS Wallet only. Transaction PIN, limits per transfer, hour and day, a 24-hour pause before money received from another user or added by card can be withdrawn, unfamiliar-device checks, SMS codes, freeze, fraud reports with reversal, and a pattern scanner. Terms Version 4.
- **Sale and inspection money**: sale proceeds are released exactly and a seller can ask for release; the cost of a physical inspection is the requester's alone and is paid through the wallet; a buyer's delivery phone number never reaches the seller.
- **Mortgage (Rent to Own)** is now held by CHS end to end (replaces the description of Rent to Own in the September pages):
    - The buyer gives their details first. Requests and the owner's answers are relayed by admin; the owner sees the buyer only as a first name and initial and never a phone number.
    - A buyer can pay one instalment or any larger amount up to the full balance. Every payment is held by CHS and released to the owner by admin (singly or Release all), with an audit trail.
    - The final payment is held until the documents are handed over: the buyer requests them, the owner uploads them for CHS to check, admin approves, the buyer receives the soft copy, the owner sends the hard copy with proof, and the buyer confirms receipt.
    - The final release is always done by hand by a super admin. The buyer's confirmation does not move money. An auto-release tick box (setting rto_final_auto_release, off by default, super admin only, audited) exists for busy periods. A super admin can reverse a release, returning the money to CHS hold. Terms Version 5.
- **Full timestamps**: every recorded event shows its date and time to the second in Nigerian time (WAT), through one shared formatter; the admin Mortgage panel shows a handover timeline. Terms Version 6 (term 37, Records and timestamps) tells users that these records are kept and may be relied on as evidence.
- **Notification bell** on every role's dashboard, opening the exact item; **newest first** everywhere except upcoming due dates and scheduled inspections.

- **Documents**: Terms Version 6, Users Guide, FAQ, Privacy Policy, Feature Catalog (PDF, spreadsheet and in-app), Handover Notes and Progress Log were brought up to date on 9 October 2026.

Statements in the September pages that are no longer true

- "No real frontend screen exists for a Rent-to-Own buyer to make an installment payment" — the buyer Mortgage page exists and works, with hold-and-release payments and the document handover.
- "Build the Rent-to-Own payment screen" listed as a next step — done.
- Any statement that a Rent-to-Own installment is credited to the owner at once — it is held and released by CHS.
- Terms described as Version 3 — the current Terms are Version 6.

Still open

- Terms Version 6 and the Privacy Policy have not been reviewed by a Nigerian lawyer.
- Earlier mortgage instalments paid under the old rule were not clawed back.
- Test data SQL for Vendor and Service Provider, and for Rent to Own, has not been written.
- Launch switches still to be set: transfer_requires_pin and withdrawal_requires_pin to true; otp_step_up_enabled to true once the SMS key works; rotate the SMS shared secret.
- Mortgage due dates, a minimum deposit and an agreement document are not built.

DOCUMENT CONTROL

System name	CHS (Complete Housing Solutions)
Document type	System Documentation (compiled reference)
Source materials	`CHS_HANDOVER_NOTES.md`, `CHS_PROGRESS_LOG.md`, chs-v2 source code (Next.js), backend-v2 SQL migrations, the live Supabase project (`havwhdgjqgtxtqklkqfm`), CHS Feature Catalog (segmented, 207 entries)
Status	Living document — reflects the state of the codebase and database as verified directly against the live Supabase project at the time of this edition. Re-verify specific counts before relying on them if significant time has passed.
Prepared for	Internal engineering handover / onboarding of a new developer or agent

1. INTRODUCTION

This document is a compiled system documentation reference for CHS (Complete Housing Solutions), assembled from the project's own real, verified handover records rather than reconstructed from general assumptions about how such a system might work. Its purpose is to give a new developer, engineer, or technical stakeholder a complete picture of what the system is, how it is built, how its pieces fit together, and what state it is honestly in — including what is proven, what is unfinished, and what has been deliberately built a certain way for reasons that are easy to miss.

Every claim of a feature being "real" or "working" in the source material reflects that it was independently tested against the live system with real data, not merely reviewed in code — a distinction the project has been careful to maintain throughout, and one this document preserves.

What changed since the last edition of this document (previously covering migrations through 222): identity verification was rebuilt from a bare ID-number field into a full, database-enforced verification required before any real commitment on the platform; rent now follows the same real escrow protection buyer and shortlet money already had; a guarantor's own details are now independently confirmed through their own private link; a security audit closed 25 database functions that had no caller check at all; and a shared, single rulebook now validates every phone number, NIN, ID, and bank account collected anywhere in the system. See Section 12 for the complete account.

2. SYSTEM OVERVIEW & BUSINESS MODEL

CHS is a real estate super-app built for the Nigerian market (developed and tested primarily around Kaduna State, with Ikeja/Lagos and other cities used in later testing rounds). It covers the full property lifecycle under one platform: buying, selling, renting, leasing, rent-to-own, short-let/hotel/event-venue bookings, property management, a building-materials and services marketplace, and artisan/maintenance job matching.

Core Business Model

CHS never allows money to move directly between two users without first passing through CHS's own wallet system. Every transaction category — sale, rental, shortlet, rent-to-own, marketplace, artisan labour — carries its own specific commission percentage, charged to one or both parties. This escrow-and-commission pattern is the single most important architectural fact about the system, and it is enforced consistently at the database function level (see Section 7).

Trust and Verification Model (substantially rebuilt this edition)

Before any real commitment — an offer, a rental application, a shortlet or hire booking, an inspection request, a rent-to-own request, a marketplace purchase or quote, or engaging CHS for a professional service — a person must complete identity verification: their name exactly as printed on a real government ID (compared automatically against their registered name for the reviewing admin), gender, age bracket, state and residential address, occupation, phone number, and email, alongside the ID document itself. This is enforced by a single shared database trigger (`enforce_verified_identity`) attached to every table where a person commits to something — not only on the relevant screen, which can be forgotten or bypassed. A National Identification Number, once verified, is permanently locked to one account.

Rental applications carry a second, independent verification layer: the applicant provides only their guarantor's name and phone number, and the guarantor completes everything else about themselves — relationship, address, occupation, their own ID, a real proof of address, and a signed consent — through their own private, single-use link, without the applicant's involvement.

Rent payments are now held in escrow — the same real protection previously exclusive to property sales — released to the landlord automatically on a genuinely clean move-in condition report, or after a 14-day grace period with nothing unresolved raised.

Deployment Footprint

Frontend hosted on Netlify, backend/database/auth/storage on Supabase, source control on GitHub. Deployment is currently a manual process: the working developer builds and packages the app, and updates are pushed to GitHub in batches, which triggers a Netlify rebuild.

3. USER ROLES

CHS supports the following distinct roles, enforced by a database check constraint. A single person may hold more than one role on a single account (e.g. Owner and Tenant simultaneously) via a "Link Account" flow, rather than maintaining separate registrations.

Role	Summary
Buyer	Browses and purchases property (outright sale or rent-to-own).
Tenant	Rents property; pays rent (held in escrow — see Section 2) and manages tenancy.
Owner	Lists property for sale/rent; manages offers, applications, and maintenance.
Agent	Manages listings and clients on behalf of owners; earns commission; can manage a team.
Property Manager	Manages estates and tenancies on behalf of owners.

Commercial Developer	Registers and manages larger-scale development listings.
Vendor	Sells building materials/products via the Marketplace.
Artisan	Offers maintenance/repair labour; quoted and paid through the platform.
Host	Lists property for short-let, hire, or event-venue booking.
Guest	Books short-let, hire, or event-venue services.
Staff/Employee	Team member under an Agent or Manager account.
Investor (new)	A restricted role for real investors/lenders doing business due diligence. Cannot reach raw user or transaction tables — sees only pre-aggregated, anonymized platform summaries (growth, financial health, revenue, geographic reach) through one narrow function. No investor accounts exist on the live system as of this edition.
Admin	Platform oversight — organised into sub-admin "domains" (`owner_buyer_tenant` , `agent_relations` , `artisan_dev_pm_vendor` , `registration_setup` , `customer_care` , `finance`) plus a super-admin role with full access and login-approval authority over every other admin.

4. SYSTEM ARCHITECTURE

Frontend

Next.js 16 using the App Router, React 19, TypeScript 5, Tailwind CSS 4. Deployed to Netlify.

Backend

Supabase (PostgreSQL + Auth + Storage + Realtime). The overwhelming majority of business logic lives in PostgreSQL functions (`SECURITY DEFINER` , called via `supabase.rpc(...)` from the client) rather than in Next.js API routes — this application has no real Next.js API routes at all. Every money-moving or state-changing operation is implemented as a single atomic Postgres function.

Request/Feature Pattern

1. A Postgres function in `backend-v2/` validates the caller via `auth.uid()` , checks identity verification where the action requires it (via the shared `enforce_verified_identity` trigger, or an explicit in-function check for admin-only functions), moves money through the `wallets` / `wallet_transactions` tables where applicable, inserts/updates the relevant tables, and calls `notify_user(...)` for any party who needs to be informed.
2. A Next.js page or component invokes this function via `supabase.rpc("function_name", { p_param: value })` .
3. Real-time or on-load queries read the resulting state directly from the relevant tables, often via a Supabase join (e.g. `properties(title, ...)`), or via a protected view where the raw table exposes more than the caller should see (see Section 7, point 10).

Authentication Pattern

Users register with a phone number. Internally, this becomes a synthetic email (`chsuser{phone}@chsplatform.app`) for Supabase Auth, while the phone number remains the human-facing identifier everywhere in the UI. PIN-based login uses this synthetic email plus a 6-digit PIN as the password. Biometric login uses WebAuthn (`@simplewebauthn/browser` and `@simplewebauthn/server`), backed by four Supabase Edge Functions. This pattern is intentional — Supabase Auth requires an email, while CHS's real identity system is phone-based — and should not be "simplified" to real emails without understanding every downstream place phone-based lookup is relied upon.

Number Validation Pattern (new)

Every phone number, National ID number, passport number, voter's card number, driver's licence number, and bank account number collected anywhere on the platform is validated against one shared rulebook, enforced in three places: live as the person types (`components/ValidatedInput.tsx`), again on form submission (`lib/validators.ts`), and — non-negotiably — again inside the database itself via a single shared trigger function (`chs_enforce_formats`), attached individually to every table that stores such a number. This exists because an earlier version of the identity-verification and guarantor forms checked only that a field was non-empty, and a client test deliberately submitting a 10-digit NIN (rather than the correct 11) was accepted.

UI Theming Pattern

Every dashboard's root element carries a `zone-{role}` class, and each role's header uses the CSS custom property `--zone-accent` (never a hardcoded colour) for its top bar. This is what gives each role's dashboard a visually distinct identity.

5. TECHNOLOGY STACK

Layer	Technology
Frontend framework	Next.js 16 (App Router), React 19, TypeScript 5
Styling	Tailwind CSS 4 — no component library; all UI hand-built
State management	Plain React state + Supabase queries — no Redux/Zustand
Backend / database	Supabase (PostgreSQL, Auth, Storage, Realtime)
Serverless functions	Supabase Edge Functions — 16 deployed (WebAuthn ceremony, Paystack webhooks/initiation, withdrawal processing, reminder dispatch; see Section 9)
Payments	Paystack (`@paystack/inline-js`) — currently test keys only
Biometric auth	`@simplewebauthn/browser` , `@simplewebauthn/server`
PDF generation	`jspdf` (client-side, where used)
Hosting	Netlify (live/current)

SMS	Termii — built, provider sender-ID approval pending
Email	Resend — built, provider approval pending

6. DATABASE ARCHITECTURE & MIGRATION HISTORY

The database schema and all business logic functions live in `backend-v2/`, as sequentially numbered, self-documenting SQL migration files. The legacy `backend/` folder (10 files) represents an earlier, pre-v2 backend attempt, superseded and not used by the live application.

Verified directly against the live Supabase project (project ref `havwhdgjqgtxtqklkqfm`) at the time of this edition:

Metric	Count
Total migrations applied	290
Tables	96
Views	3
Functions	443
Triggers	35
RLS policies	260
Scheduled (cron) jobs	5
Storage buckets	`property-media` (public), `marketplace-media` (public), `private-documents` (private — ID documents, guarantor proofs, liveness captures)

Migrations group into the following honest phases:

Range	Phase
1–50 (approx.)	Core schema — profiles, properties, wallets, offers, tenancies, original registration/KYC system, basic commission logic.
50s–90s	Bug-fix rounds following early client testing — dispute rulings with no real consequence, agent referral payouts that never fired, management termination that never worked, developer approval with no real consequence, maintenance reserve never wired up.
90–150	Feature rounds — Rent-to-Own, shortlet messaging/escalation, sale legal-document + escrow flow, agent full-authority management, precommit message moderation, estate management, team subscriptions, KYC fixes, property reference numbers, PIN reset, FAQ, per-role colour themes.
150–195	Team/business tools for agents and managers, Host/Guest/Developer as separate roles, complete shortlet/hire rebuild, receipt/voucher system, tiered artisan commission.
196–286	Complete marketplace rebuild, rental-application rework with independent guarantor verification, Event Centre pricing, payment-regression fixes, "Recently Handled" pattern rolled out across every admin queue, audit trail, transaction history log, refund handling, investor role, user reference numbers.
287–410 (most recent)	Real-device client testing round: duplicate/blocked rent-payment fixes traced and reversed, guarantor upload/permission fixes, splash-screen root cause found and fixed (traced through three separate, real regressions), rent escrow built and completed with automatic release, identity verification fully rebuilt, one shared number-validation rulebook, a security audit closing 25 unguarded database functions, owner-safe protected views for applications and offers.

> **Guidance for any future work:** read migration filenames in numeric order — each carries its own top comment explaining the real problem it fixes or the feature it adds. Never renumber or delete an existing migration file; add new ones sequentially. Before making any schema change, confirm the live Supabase migration count still matches the highest number recorded here. **Known gap, stated plainly:** migrations 196–382 in the `backend-v2/` folder currently contain only summary comments, not runnable SQL — the live database is correct and complete, but the exportable folder cannot currently rebuild that range from scratch. See Section 8.

Edge Functions

16 Supabase Edge Functions are deployed. Four implement the complete WebAuthn ceremony for biometric login (`webauthn-register-options`, `webauthn-register-verify`, `webauthn-auth-options`, `webauthn-auth-verify`). The remainder handle Paystack wallet funding and webhooks, withdrawal initiation and processing, and multi-channel reminder dispatch.

7. KEY DESIGN DECISIONS (things a new developer might otherwise "fix" incorrectly)

1. Every payment computes and charges its own commission within the same atomic function call — never as a separate step a different function is trusted to remember later. This was violated twice early on (`pay_rent`, `pay_rent_to_own_installment`) and caused a real client-facing bug before being caught and fixed. `pay_for_property` (Sale) was always correct and is the reference pattern to copy.
2. Guarantor details on a rental application must never be entered by anyone but the guarantor. The applicant provides only a name and phone number; the guarantor completes everything else — relationship, address, occupation, their own ID, a real proof of address, and a typed-name consent — through their own private, single-use link. This was deliberately split apart after a direct client concern about consent and fraud risk.
3. The marketplace intentionally has two separate purchase paths: a moderated quote-request negotiation, and a direct "Buy Now" for fixed-price items. Both use the same commission split and the same escrow mechanism.
4. `transaction_commissions.status` defaults to `'pending'` and must be explicitly set to `'paid'` the moment money actually moves. A stale-commission alert (Admin → Finance) exists specifically to catch any future function that inserts a commission row and forgets this step.
5. Contact-info blocking (`detect_offplatform_contact`) is a shared, reusable function, used identically across property-offer negotiation and the marketplace.
6. Notification links matter as much as the notification itself. `notify_user(p_user_id, p_title, p_body, p_link)` — the 4th parameter (link) has

repeatedly been forgotten. A notification without a working link to the relevant page is treated as a reportable bug, not a nice-to-have.

7. Document URLs must never be trusted long-term. `getFreshDocumentUrl()` in `lib/storage.ts` generates a fresh signed URL at the moment a document is actually viewed.
8. The synthetic-email auth pattern (`chsuser{phone}@chsplatform.app`) is intentional, not a hack to clean up — see Section 4, Authentication Pattern.
9. **(New)** Identity verification is enforced at the database, not only on screen. The shared trigger `enforce_verified_identity` is attached directly to the table a person is about to commit to (offers, rental applications, shortlet/hire bookings, inspections, rent-to-own, marketplace orders/quotes, Engage CHS requests), so the rule cannot be bypassed by a new screen that forgets to check, or a direct API call.
10. **(New)** A counterparty is never handed another user's raw record. An owner reading a rental application or offer through the API previously received every column — the applicant's NIN, ID document link, both phone numbers, the guarantor's ID and confirmation token — even though the screen displayed less. Two protected views (`owner_rental_applications` , `owner_offers`) now give an owner only the fields their own screens genuinely use. **This fix is only half-complete** — see Section 8.
11. **(New)** Every phone/ID/NIN/bank-account number is validated by one shared rulebook, in three places at once (typing, submission, database) — see Section 4, Number Validation Pattern. Never add a bespoke check for a new number field; extend the shared rulebook instead.
12. **(New)** A privileged (`SECURITY DEFINER`) database function must check who is calling it. A security audit found 25 functions — including wallet credit primitives and admin-only data feeds — with full database rights and no caller check at all, reachable by anyone including people who were not signed in. When adding a new `SECURITY DEFINER` function, decide explicitly whether it is public, signed-in-only, admin-domain-only, or server-only, and enforce that inside the function itself.

8. KNOWN ISSUES & LIMITATIONS

Confirmed, fixed, and proven this edition (documented so they are not re-discovered)

- A duplicate real rent payment and a bug blocking a brand-new tenancy's very first payment — both traced to the same 30-day "not yet due" rule, applied too broadly at first, then correctly scoped to renewals only.
- A tenant able to pay a rental commission alone, with the real rent never moving — traced to an earlier fix wiring the generic sale-commission function onto rentals. Fixed at the source: a rental's first commission now routes through `pay_rent` , which correctly combines both.
- The splash-screen/reload complaint, open across several prior rounds — traced through three separate, real causes in sequence (a service worker caching full pages instead of only static files; then, after that was removed entirely, a notification-click handler opening links in a literal new browser tab, which cannot avoid a full cold start). Fixed decisively: the service worker was removed outright, not patched a third time.
- Rent moved directly to the landlord's spendable balance the instant it was paid, with no protection — while sale and shortlet money were already correctly held in escrow. Rent now follows the identical real pattern.
- The promised automatic release of held rent after its grace period was never actually scheduled. A daily cron job now performs this, holding automatically if a real fault or a flagged condition-report item is still open.
- 25 privileged database functions had no caller check at all, confirmed reachable by an unauthenticated caller (one returned 35 real registration records). Closed at the database level.

Confirmed but not yet fixed — genuinely open items

- **The owner data-protection fix is only half-applied.** Protected views now exist and the owner screens use them, but the old, unrestricted direct table access is deliberately still open, pending confirmation from the client that the owner screens work correctly on this release. Two RLS policies need dropping once confirmed (see migration 401_403's own header for the exact commands).
- **A real secret is embedded directly in the `send-multichannel-reminder` edge function's source** and was exposed during this audit. Should be treated as compromised and rotated in the Supabase dashboard.
- **No privacy policy existed anywhere until this edition**, despite the platform collecting name, gender, age, address, occupation, email, phone, and government ID at verification. A first draft now exists at `/privacy` and should be reviewed by a Nigerian lawyer familiar with the NDPA before being treated as legally final.
- `backend-v2/` **migrations 196–382 contain only summary comments, not runnable SQL.** The live database is the authoritative, correct record in the meantime; this needs a dedicated export pass, not further hand-transcription.
- No real frontend screen exists for a Rent-to-Own buyer to make an installment payment through the UI. The backend function is correct and tested directly.
- No real recurring billing exists for any promotion subscription package (Classic/Premium/Elite/Signature). The pricing and database rows exist; the actual Paystack subscription charge was never built.

Structural gaps

- No automated test suite exists anywhere in the project. All verification is manual — direct Postgres function calls against real or constructed test data, checking before/after state, plus `npm run lint` and `npm run build` for frontend changes.
- No CI/CD pipeline. Deployment is manual.
- No uptime or error monitoring, and Supabase backup/point-in-time recovery configuration has never been directly confirmed.

9. EXTERNAL SERVICES & INTEGRATIONS

Service	Used for	Status
Supabase	Entire backend — database, auth, storage, real-time, Edge Functions, scheduled jobs	Live, active (project ref: `havwhdgjqgtxtqklkqfm`)
Netlify	Frontend hosting/deployment	Live, active
GitHub	Source control	Live, active — manual push of zip updates
Paystack	Wallet funding / real money movement	Test keys only — real payments not yet live

Termii	SMS (OTPs, rent reminders, etc.)	Built; sender-ID approval pending
Resend	Transactional email	Built; provider approval pending

No other third-party APIs are in use, confirmed by codebase search: no Google Maps, no third-party NIN verification API, no third-party face-liveness service. NIN uniqueness and format are checked only within CHS's own database (Section 4), and identity verification is a document and photo reviewed manually by CHS staff.

10. SECURITY CONSIDERATIONS

A direct security audit was carried out this edition, described honestly here rather than only in general terms.

Findings, and their current state:

- **25 database functions with full privileges had no caller check at all**, executable by anyone including unauthenticated callers — confirmed directly, not assumed: an unauthenticated call to one admin data feed returned 35 real registration records including ID document links. **Fixed** — internal functions (wallet credits, scheduled jobs) are now locked to the server only; every admin data feed checks the caller's real staff domain; a listing-promotion function no longer accepts a negative amount or someone else's wallet.
- **An owner could read a rental applicant's or buyer's complete raw record** — NIN, ID document link, both phone numbers, guarantor ID and confirmation token — through the API, regardless of what the screen displayed. **Half-fixed** — protected views exist and the screens use them; the old, unrestricted access is deliberately still open pending confirmation (see Section 8).
- **A real secret embedded in an edge function's source** was exposed during the audit. **Not yet rotated** — see Section 8.
- **A public, internal test-results table had no row-level security. Fixed** — RLS enabled, admin-only read policy added.

Standing security measures, unchanged and still correct:

- Row-Level Security (RLS) policies gate access throughout the database, layered on top of the `SECURITY DEFINER` function pattern described in Section 4.
- Private, sensitive documents are served exclusively via freshly generated signed URLs (`getFreshDocumentUrl()`) rather than stored URLs, to survive signing-key rotation without silent breakage.
- Contact-information leakage between negotiating parties is actively detected and blocked (`detect_offplatform_contact`).
- Biometric login uses the standard WebAuthn ceremony rather than a custom implementation.
- No API keys, passwords, or service-role secrets are recorded in this document or any of its source material.

Two `SECURITY DEFINER` views (`owner_rental_applications` , `owner_offers`) show as ERROR-level in Supabase's own advisor. This is correct by design — the entire reason for building them was to bypass RLS safely inside a controlled function, exposing only a deliberately narrow set of columns, and is not a finding to act on.

11. ENVIRONMENT & DEPLOYMENT

Local development setup

`cd chs-v2 && npm install`

Create `chs-v2/.env.local` (never commit) with:

```
NEXT_PUBLIC_SUPABASE_URL=<the real Supabase project URL>
NEXT_PUBLIC_SUPABASE_ANON_KEY=<the real Supabase anon/public key>
```

Then: `npm run dev` (local server) · `npm run build` (must be run before considering any change complete) · `npm run lint` (should show 0 errors).

Database migrations

Add a new, sequentially numbered `.sql` file to `backend-v2/` (check the highest existing number first) and apply it directly to the live Supabase project. Never renumber or delete an existing migration file.

Deployment

Frontend deploys to Netlify from GitHub. There is no CI/CD pipeline — pushes are manual and batched.

12. RECENT CHANGE HISTORY (most recent round, in order)

A real-device client testing round, dominated by bugs found through sustained direct use rather than code review. Full detail in `CHS_PROGRESS_LOG.md` ; this is the compiled summary.

1. A duplicate real rent payment traced and reversed; the same underlying rule then found to wrongly block a brand-new tenancy's first-ever payment, fixed properly.
2. Guarantor ID upload hardened — three separate real storage/permission bugs found and fixed in sequence, plus a required address-proof document and a same-surname block.
3. The splash-screen/reload complaint, open across several rounds, traced through three separate real causes and fixed decisively (service worker removed outright, not patched again).
4. A tenant able to pay a rental commission with the real rent never moving — a genuine self-introduced regression, found, owned directly, and fixed at the source.
5. A complete rental-data reset performed on direct instruction, with every real wallet impact reversed and calculated (not guessed) before deletion.
6. Rent escrow built: held rent, released on a clean move-in report or after a 14-day grace period, with an automatic daily release job added after being found missing.
7. Identity verification rebuilt in full — name comparison, gender, age bracket, state, address, occupation, phone, email — made mandatory before

any commitment anywhere on the platform, enforced at the database.

8. One shared rulebook built for every phone, NIN, ID, and bank-account number collected anywhere, after a client test exposed that only "not empty" was being checked.
9. A real data-protection gap found and half-closed: protected views now limit what an owner can read about an applicant or buyer; the old unrestricted access remains open pending confirmation (see Section 8).
10. A broader security audit, done unprompted while writing documentation: 25 unguarded privileged functions closed; one exposed secret flagged for rotation.
11. Upload boxes redesigned everywhere as real drag-and-drop targets; a "Take a Tour" photo/video gallery built for every property; a new "Escrow Oversight" admin tab combining every category of held money into one view.
12. Documentation brought current: Progress Log, Handover Notes, Terms & Conditions (four new clauses), a Privacy Policy written from scratch, the Users Guide, the Feature Catalog (PDF and XLSX), and this System Documentation — all regenerated directly from their real, current sources rather than left to drift.

13. PENDING WORK (prioritised)

1. **Confirm the owner screens work correctly against the new protected views**, then drop the two remaining unrestricted-access RLS policies named in Section 8 — the single highest-priority open item.
2. **Rotate the exposed secret** in the `send-multichannel-reminder` edge function.
3. **Have a Nigerian lawyer review the new Privacy Policy** (`/privacy`) before treating it as legally complete.
4. Export and correct the `backend-v2/` migration backlog for range 196–382, currently comment-only.
5. Build the Rent-to-Own payment screen. Backend is complete and proven; this is frontend-only work.
6. Complete SMS (Termii) and email (Resend) provider approval.
7. Move Paystack off test keys once business verification is complete.
8. Regenerate the CHS Fix Tracker from a real, current source of tracked items — deliberately not attempted blind this edition, since it is a structured status table, not prose, and has no editable source.
9. The company website (`website/index.html`) is not live and was explicitly held back from this round of work pending direct client instruction.
10. Maintain the established discipline: whenever any payment flow is touched, walk it end-to-end with real data and a real resulting receipt before considering it done.

APPENDIX A — ROUTE MAP ADDITIONS THIS EDITION

The complete route map from the previous edition remains accurate and is not reproduced in full here to avoid duplicating over 60 unchanged entries. The following routes are new since migration 222:

Route	Purpose
/privacy	**New.** The real Privacy Policy, linked from the homepage footer, the More menu, and directly beside the consent checkbox on `/accept-terms` .
/guarantor-confirm/[token]	The independent, no-login-required page a rental guarantor uses to confirm their own details, ID, proof of address, and consent — now with real, validated ID-number and phone inputs.
/my-rented-space	A tenant's single real home for their active tenancy — countdown, both wallets, rent payment (now showing the real combined rent+commission total, held-in-escrow status), fault log, condition reports, and direct messaging to whoever is actually responsible (manager if delegated, landlord otherwise).
/condition-report/[tenancyId]	Rebuilt this round — rooms auto-populate from the property's real bedroom count, a standard fixture checklist per room, a photo field per fixture, and a real View link (PDF) ahead of the Download link for the standard affidavit template.
/admin (Escrow Oversight tab)	**New.** One combined view of every category of money currently held — property sale escrow, marketplace escrow, shortlet/hire deposits, and held rent — with a live combined total and manual early-release for rent.

APPENDIX B — NOTE ON THE FEATURE CATALOG

The complete, current feature-by-feature catalog (207 entries across 15 sections, cross-referenced to where each feature lives and how admin oversees it) is deliberately not reproduced inline in this document. It is maintained as its own real, structured data source (`types/featureCatalogData.ts`), which directly powers three things kept in guaranteed sync with each other: the in-app catalog at `/admin/feature-catalog` , the standalone `CHS_COMPLETE_FEATURE_CATALOG.pdf` , and `CHS_FEATURE_CATALOG_SEGMENTED.xlsx` . Duplicating all 207 entries a fourth time, inline in this document, would only create a fourth copy that could quietly drift out of date — the earlier edition of this document did exactly that (193 entries, frozen at whatever the count was on the day it was written). Consult the Feature Catalog directly; it is the real, current source, not this document.

Update, 14 October 2026

This addendum brings the System Documentation up to date. Read it with section 12 (recent change history), section 7 (key design decisions) and section 10 (security).

12 (continued). What was built, in order

1. Event-centre advance: 60% of the venue's net earnings (booking price less service charge) released about 48 hours before the event; balance after. Same for services. Settings event_advance_pct (0 to 90) and event_advance_release_hours.
2. Clawback on cancel and refund; any shortfall stored in advance_recover_due with an admin alert. CHS-sourced providers have advance_due and advance_status on service_provider_payouts.
3. Event risk class, private booking documents (bucket private-documents), licensed capacity with admin verification and a database trigger, venue profile, seating layouts and capacity per layout.
4. Hotels: several rooms per request (room_group, room_no, room_count), adults and children, pay_room_group, stay extras, public rating summary.
5. Search by dates (properties_free_for_dates), accessibility tags, weekly and monthly discounts, group enquiries, open_field and stadium categories with open_venue_profile.
6. VAT recording: tax_ledger and triggers; settings vat_enabled (false), vat_rate_pct, vat_tin, vat_registered_since; admin_get_vat_settings, admin_save_vat_settings, admin_get_vat_report.
7. Security: execute revoked from anonymous and public on 18 new functions (migration 508).

7 (continued). Design decisions

1. Money rules live in security-definer database functions. The advance is released only by the sweep or by an admin, and every cancel path takes it back first.
2. VAT is recorded as included in CHS's fee; no amount a user pays changes. Adding VAT on top is a separate, deliberate change.
3. Tables reached only through functions (booking_documents, group_enquiries) have row security on and no policies, on purpose.
4. Free text that guests or hosts type is passed through detect_offplatform_contact before it is stored.
5. New functions: revoke from public and anon, grant to authenticated, unless they are deliberately public.

8 (continued). Known issues

- Terms V9 not active and not lawyer-reviewed.
- Platform earnings tab excludes service fees.
- Change-request answers and host-initiated chat skip the admin relay.
- Host calendar grid and iCal do not show event sessions.
- Rain plan and crowd limit on open grounds are information only.
- Seating diagrams are illustrations, not a hall plan.
- Security advisor (14 Oct): 2 security-definer views, 248 functions without a fixed search path, 306 anonymous-callable privileged functions, 2 extensions in the public schema, leaked-password protection off. See the Security Audit update.

13 (continued). Pending work

1. Lawyer review of Terms V9; activate it with CURRENT_TERMS_VERSION = 9 and the activation SQL.
2. Accountant to confirm VAT registration, rate and exemptions; then switch VAT recording on.
3. Batch-revoke anonymous access on older functions; fix the two views; switch on leaked-password protection.
4. Live test of the advance: booking 018429ef is already paid, so press Release advance now in Admin, Event Centres.
5. Launch items: lawyer, CAC, launch switches, SMS secret rotation, WhatsApp provider, Netlify redirect URLs, WebAuthn and Paystack for the new address.

Update 2, 14 October 2026: managing hotels, event centres and open venues

- Management requests are now open to commercial hire properties from the Host and Owner pages. The database requires proof of ownership, the operating permit or licence, public liability insurance and, for event centres and open venues, a fire safety certificate.
- On a CHS-managed hotel, event centre or shortlet the host-side booking commission is not charged (trigger managed_no_host_commission_t, function _chs_managed_hospitality, pricing patched). The guest-side fee and service fees are unchanged.
- Management terms version 3 adds section 6A (event centres, halls and open venues).
- Open: no expiry tracking on the permit, fire and insurance documents; renewal-year rent commission on managed rentals unchanged; lawyer review.